

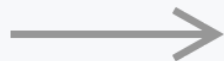
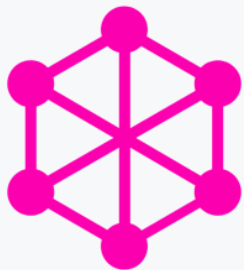
FEDERATED VALIDATION

WHAT I'M TALKING ABOUT

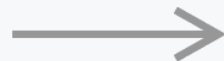
- What is validation
- Validation of a single service
- Validation of federated services
- What validation errors look like

WHAT IS VALIDATION?

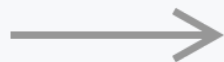
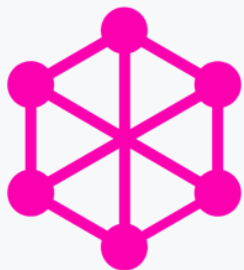
SINGLE SERVICE



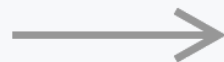
validate



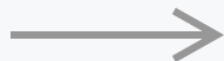
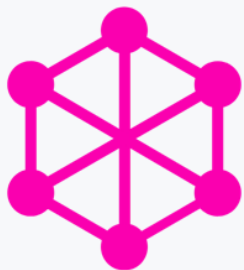
GraphQLError[]



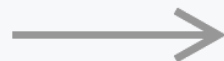
validate



GraphQLError[]

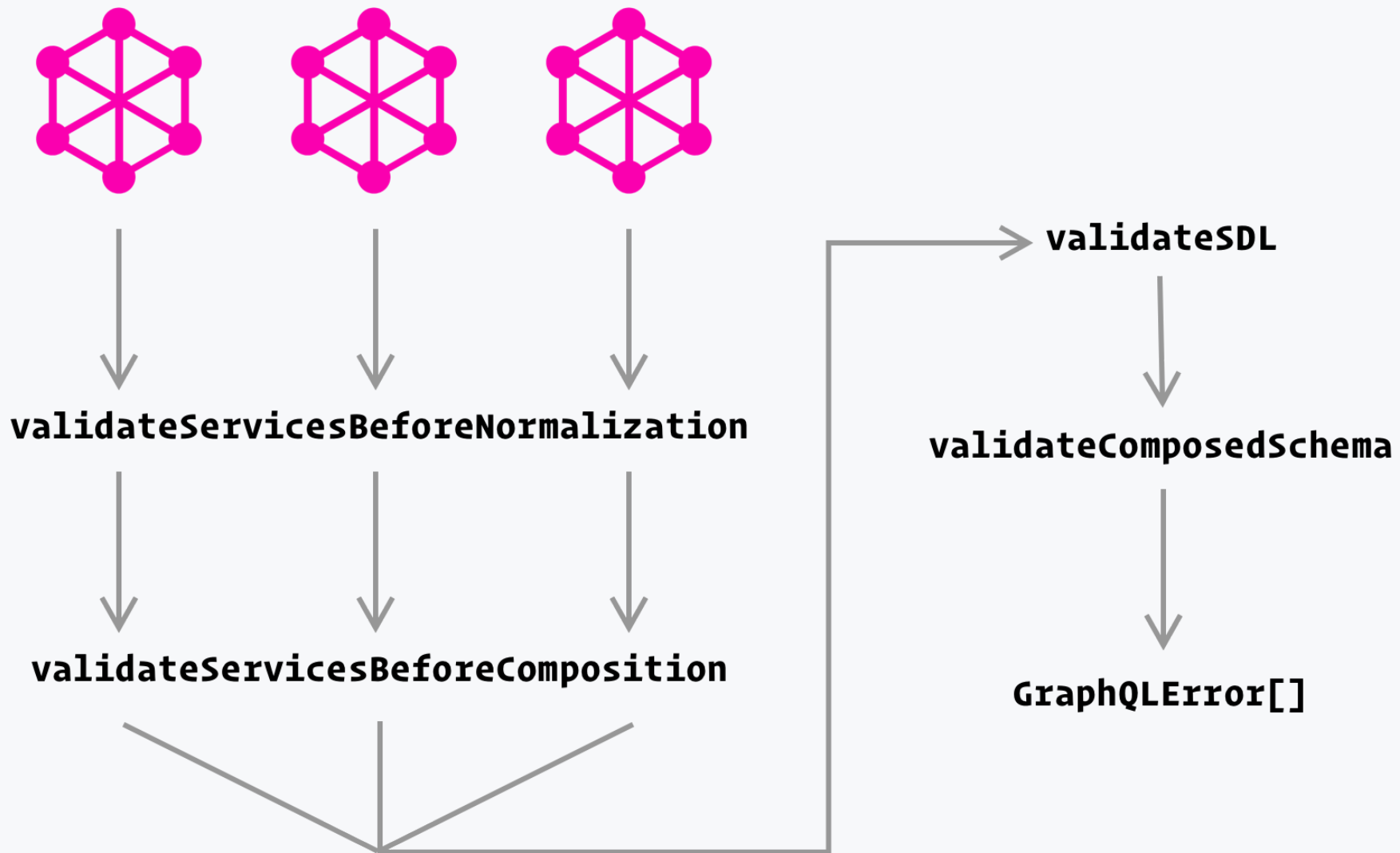


validate



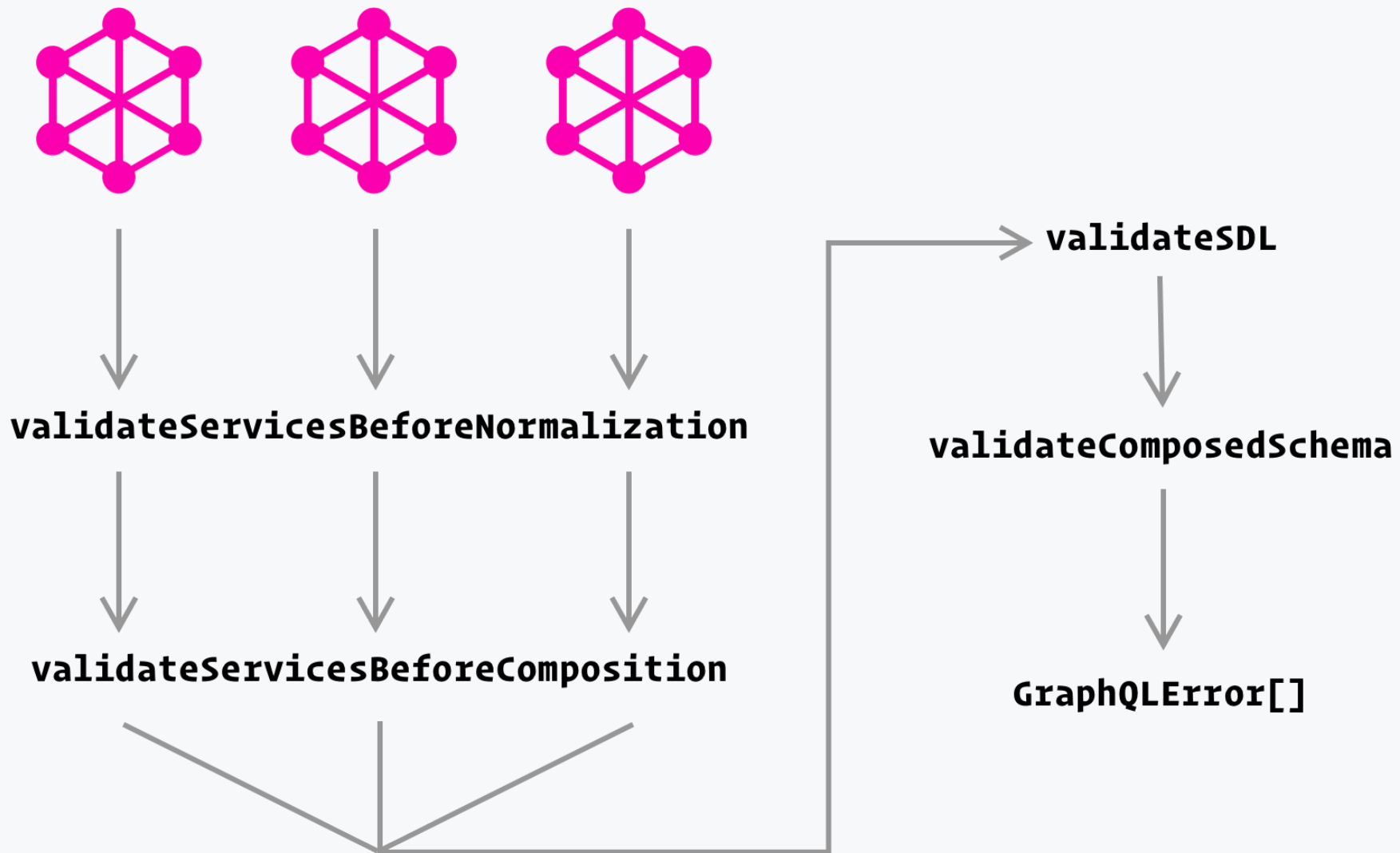
GraphQLError[]

FEDERATED SERVICES



WHY DOES IT LOOK LIKE THIS?

- It's hard to know what the original schemas looked like.
- Types are merged with extensions
- Certain fields are stripped (like `@external` fields)



```
export function composeAndValidate(serviceList: ServiceDefinition[]) {  
  const errors = validateServicesBeforeNormalization(serviceList);  
  
  const normalizedServiceList = serviceList.map(({ name, typeDefs }) => ({  
    name,  
    typeDefs: normalizeTypeDefs(typeDefs),  
  }));  
  errors.push(...validateServicesBeforeComposition(normalizedServiceList));  
  
  const compositionResult = composeServices(normalizedServiceList);  
  errors.push(...compositionResult.errors);  
  
  errors.push(...validateComposedSchema(compositionResult.schema));  
  
  return { schema: compositionResult.schema, warnings: [], errors };  
}
```

validateServicesBeforeNormalization

- Normalization makes sure the root operation types (Query and Mutation) are named **Query** and **Mutation**, as well as handling the **@extends** directive.
- This function validates things that would cause errors in normalization

RULES (SO LONELY).

- `rootFieldUsed`
 - If the base types are renamed, make sure `Query` and `Mutation` aren't used.

validateServicesBeforeComposition

- Validates each schema separately
- Composition modifies the schemas and strips out things like **@external** fields, so we need to validate the things that get stripped

RULES

- `externalUsedOnBase`
 - `@external` directive can not be used on base types
- `requiresUsedOnBase`
 - `@requires` directive can not be used on base types
- `keyFieldsMissingExternal`
 - `@key` selects field not marked as `@external`
- +3 more

validateSDL

- Validated *during* composition.
- Uses list of all type definitions/extensions
- Uses some **graphql-js** validations
 - omits rules that would conflict with federation spec like **UniqueTypeNames**
- Additional validations related to cross-service type definitions

RULES

- **UniqueTypeNamesWithoutEnumsOrScalars**
 - Modified version of **UniqueTypeNames**
- **MatchingEnums**
 - Enums must be identical in each service
- **PossibleTypeExtensions**
 - Type extensions must extend existing types

`validateComposedSchema`

- Preferred place to validate if possible
- Validates the result of schema composition
- Uses some validation rules from graphql-js

RULES

- `externalUnused`
 - external field isn't used
- `externalMissingOnBase`
 - external used on a field not on the base type
- `externalTypeMismatch`
 - external field doesn't match the field on the base type
- +6 more

ANATOMY OF AN ERROR

```
// Service A
type Product @key(fields: "sku") {
  sku: String!
  upc: String!
}
```

```
// Service B
extend type Product @key(fields: "sku") {
  sku: String @external
  price: Int! @requires(fields: "sku")
}
```

```
GraphQLError {  
  message: `  
    [serviceB] Product.sku ->  
    Type `String` does not match the type of  
    the original field in serviceA (`String!`)  
  `,  
  extensions: {  
    code: 'EXTERNAL_TYPE_MISMATCH'  
  }  
}
```

Apollo Server

Version 2 ▾

[Introduction](#)[Getting started](#)[What's new?](#) COLLAPSE ALL

ESSENTIALS

[Understanding schema concepts](#)[Building a server](#)[Fetching data with resolvers](#)

FEATURES

[Caching](#)[Mocking](#)[Error handling](#)[Data sources](#)[Subscriptions](#)[Monitoring and metrics](#)[GraphQL Playground](#)[Custom scalars and enums](#)[Unions and interfaces](#)[Using schema directives](#)[Implementing directives](#)[Authentication](#)

Understanding errors

Rules for schema composition

Apollo Federation implements a strict composition model. When building a gateway using the `@apollo/gateway` package, `gateway.load()` validates that the provided services compose into a valid GraphQL schema **and** that the gateway has all of the information it needs to execute operations against the composed schema. Any errors will fail composition. This section documents the composition errors that can be thrown by `gateway.load()`.

Types and extensions

- `EXTENSION_OF_WRONG_KIND`: An extension is attempting to extend a different type. For example, an interface extension, `extend interface MyType`, would be invalid for a type definition such as `type MyType`.
- `EXTENSION_WITH_NO_BASE`: An extension is attempting to extend a type which does not exist.

@key directives

- `KEY_FIELDS_SELECT_INVALID_TYPE`: The fields argument can not have root fields that result in a list, interface, or union type.
- `KEY_FIELDS_MISSING_ON_BASE`: the fields argument can not select fields that were overwritten by another service.
- `KEY_FIELDS_MISSING_EXTERNAL`: On extended types, keys must reference a field marked as `@external`.

Understanding errors

 [Edit on GitHub](#) [Discuss on Spectrum](#)

```
"Loading Apollo Project [started]
Loading Apollo Project [completed]
Validate graph composition for service accounts on graph engine [started]
→ Fetching local service's partial schema
→ Attempting to compose graph with accounts service's partial schema
Found 3 graph composition errors for service accounts on graph engine [title changed]
Found 3 graph composition errors for service accounts on graph engine [failed]
→ Federated service composition was unsuccessful. Please see the reasons below.
```

Service	Field	Message
reviews	User.id	marked @external but it does not have a matching field on on the base service (accounts)
reviews	User	A @key selects id, but User.id could not be found
accounts	User	A @key selects id, but User.id could not be found

